

MSS-Assignment II

313512009 施丞蔚

2.Cachegrind - [10%]

Requirement: Please take screenshots of two logs and point out where the difference is and explain why this problem occurs in the report.

MSS > HW2 > 313512009_good_log

```
1 ==27604== Cachegrind, a cache and branch-prediction profiler
2 ==27604== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
3 ==27604== Using Valgrind-3.20.0 and LibVEX; rerun with -h for copyright info
4 ==27604== Command: ./good
5 ==27604== Parent PID: 5181
6 ==27604==
7 --27604-- warning: L3 cache found, using its data for the LL simulation.
8 --27604-- warning: specified LL cache: line_size 64  assoc 16  total_size 12,582,912
9 --27604-- warning: simulated LL cache: line_size 64  assoc 24  total_size 12,582,912
10 ==27604==
11 ==27604== I   refs:      30,156,482
12 ==27604== I1  misses:      1,103
13 ==27604== L1i misses:      1,091
14 ==27604== I1  miss rate:      0.00%
15 ==27604== L1i miss rate:      0.00%
16 ==27604==
17 ==27604== D   refs:      14,053,454 (12,039,353 rd + 2,014,101 wr)
18 ==27604== D1  misses:      127,181 (   64,090 rd +   63,091 wr)
19 ==27604== L1d misses:      64,332 (    1,290 rd +   63,042 wr)
20 ==27604== D1  miss rate:      0.9% (    0.5% +    3.1% )
21 ==27604== L1d miss rate:      0.5% (    0.0% +    3.1% )
22 ==27604==
23 ==27604== LL refs:      128,284 (   65,193 rd +   63,091 wr)
24 ==27604== LL misses:      65,423 (    2,381 rd +   63,042 wr)
25 ==27604== LL miss rate:      0.1% (    0.0% +    3.1% )
```

MSS > HW2 > 313512009_bad_log

```
1 ==27659== Cachegrind, a cache and branch-prediction profiler
2 ==27659== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
3 ==27659== Using Valgrind-3.20.0 and LibVEX; rerun with -h for copyright info
4 ==27659== Command: ./bad
5 ==27659== Parent PID: 5181
6 ==27659==
7 --27659-- warning: L3 cache found, using its data for the LL simulation.
8 --27659-- warning: specified LL cache: line_size 64  assoc 16  total_size 12,582,912
9 --27659-- warning: simulated LL cache: line_size 64  assoc 24  total_size 12,582,912
10 ==27659==
11 ==27659== I   refs:      30,156,482
12 ==27659== I1  misses:      1,103
13 ==27659== L1i misses:      1,091
14 ==27659== I1  miss rate:      0.00%
15 ==27659== L1i miss rate:      0.00%
16 ==27659==
17 ==27659== D   refs:      14,053,454 (12,039,353 rd + 2,014,101 wr)
18 ==27659== D1  misses:      2,002,182 ( 1,001,591 rd + 1,000,591 wr)
19 ==27659== L1d misses:      64,333 (    1,291 rd +   63,042 wr)
20 ==27659== D1  miss rate:     14.2% (    8.3% +   49.7% )
21 ==27659== L1d miss rate:      0.5% (    0.0% +    3.1% )
22 ==27659==
23 ==27659== LL refs:      2,003,285 ( 1,002,694 rd + 1,000,591 wr)
24 ==27659== LL misses:      65,424 (    2,382 rd +   63,042 wr)
25 ==27659== LL miss rate:      0.1% (    0.0% +    3.1% )
```

項目	good	bad	差異解釋
D1 misses	127,181 次 (0.9%)	2,002,182 次 (14.2%)	差異非常大，說明 bad 使用的資料不具區域性（locality），導致快取無法有效命中。
D1 misses 讀取 (rd)	64,090 次 (0.5%)	1,001,591 次 (8.3%)	bad 的讀取分散，導致資料每次幾乎都要從記憶體撈
D1 misses 寫入 (wr)	63,091 次 (3.1%)	1,000,591 次 (49.7%)	bad 在資料寫入上完全不具備區域性，浪費大量快取資源
LL misses	65,423 ($\approx 0.1\%$)	65,424 ($\approx 0.1\%$)	長期來看兩者都能保持低 LL miss，但 bad 對快取壓力大
總快取存取 (LL refs)	128,284 次	2,003,285 次	bad 因 L1 命中率低，需要不斷訪問下層快取
I1 misses	1,103 次	1,103 次	指令快取相同，說明差異主要來自資料快取

1. 差異 1：D1 misses 總數

- good: 127,181 次
- bad: 2,002,182 次（高出約 15 倍）

說明：

D1 cache 是 L1 層級的資料快取，速度快但容量小。

- good 在大部分資料讀寫上都命中 cache，表示資料具有高區域性（spatial & temporal locality）。
- bad 幾乎每筆資料都 miss，表示其存取順序極差、資料分散，導致快取難以保存有效內容。
- 程式可能使用了不良的資料結構（如跳躍式索引、亂序迴圈、pointer chasing）。

2. 差異 2：D1 讀取 miss (rd)

- good: 64,090 次 → miss rate 0.5%
- bad: 1,001,591 次 → miss rate 8.3%

說明：

- 在 good 程式中，大部分的讀取是從快取中直接完成。
- 而 bad 的讀取 miss 超過百萬次，表示其讀資料的順序與快取存放邏輯不一致。
- 原因可能是：訪問跨越陣列的行而非列 (row-major vs column-major)、或不連續的指標跳躍式讀取。

3. 差異 3：D1 寫入 miss (wr)

- good: 63,091 次 → miss rate 3.1%
- bad: 1,000,591 次 → miss rate 49.7%

說明：

- 寫入 cache 的資料通常會觸發 write allocate (先讀入 cache 再寫入)，因此快取行為更敏感。
- bad 的寫入 miss 幾乎達 50%，表示其寫入資料完全不具局部性。
- 原因可能是：每次寫入都是不同區域的記憶體 (例如跳過 cache line 大小的間隔) 或每次覆蓋的資料都未事先預讀 (pre-fetch)

4. 差異 4：LL (Last-Level Cache) References

- good: 128,284 次
- bad: 2,003,285 次 (高出超過 15 倍)

說明：

- LL refs = 無法在 L1 找到 → 往 L2/L3 cache 查找。
- bad 的 L1 miss 率高，導致不得不往更慢的 cache 層查資料。
- 即使 LL miss 數相對接近，但高頻次訪問 LL cache 會拖慢整體速度。

5. 差異 5：LL misses (LLd + LLi)

- good: 65,423
- bad: 65,424

結論：兩者 LL miss 差不多

- 表示最終還是能命中 L2/L3 cache，但：
- bad 需要更多次地「穿越」L1 → LL
- good 則大部分資料留在 L1 中即可命中
- 這造成 bad 雖然 LL miss 不高，但整體效率被 L1 miss 拖垮。

6. 差異 6：D1 miss rate（總體）

- good: 0.9%
- bad: 14.2%

說明：

這算是最具代表性的指標之一，代表「每 100 次資料讀寫，就有幾次需要進一步存取下層快取」。

- good 表現良好，表示設計上有刻意優化資料存取順序。
- bad 表現非常差，基本等於「每次存取幾乎都要走遠路」。

7. 差異 7：LL miss rate

- good: 0.1%
- bad: 0.1%（略高但相近）

說明：

- 說明兩份程式對於 L2/L3 cache 使用都還在合理範圍內。
- 性能瓶頸主要來自 L1 cache（D1）無法命中。

整體結論（可寫入報告摘要）

本次分析比較 good 與 bad 程式的 cache 行為，發現兩者在 L1 data cache（D1）命中率上有顯著差異：

- good 在資料讀寫上具有良好區域性，D1 miss rate 僅 0.9%。
- bad 的資料存取缺乏區域性，D1 miss rate 高達 14.2%，尤其寫入 miss 接近 50%。
- 雖然兩者在 L2/L3 cache（LL）表現相近，但因 bad 大量無效率的 L1 存取導致系統頻繁 fallback 到下層 cache，拖慢整體效能。

建議優化 bad 程式的資料存取模式，例如：

- 優化迴圈順序
- 使用陣列連續存取
- 避免 pointer chasing 或不連續寫入

3. Massif - [10%]

1.[5%] Please observe the relationship between time and memory allocation throughout the entire program execution, and provide one screenshot of the output file containing relevant information. (You must clearly display the total number of snapshots each time the system records the information in intervals).

n	time(B)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
50	183,488	183,488	183,000	488	0
51	191,496	191,496	191,000	496	0
52	193,504	193,504	193,000	504	0
53	197,512	197,512	197,000	512	0
54	201,520	201,520	201,000	520	0
55	203,528	203,528	203,000	528	0
56	207,536	207,536	207,000	536	0
57	211,544	211,544	211,000	544	0
58	219,552	219,552	219,000	552	0
59	221,560	221,560	221,000	560	0
60	225,568	225,568	225,000	568	0
61	229,576	229,576	229,000	576	0
62	231,584	231,584	231,000	584	0
63	235,592	235,592	235,000	592	0
64	239,600	239,600	239,000	600	0
65	239,600	239,600	239,000	600	0
66	240,616	238,584	238,000	584	0
67	241,632	237,568	237,000	568	0
68	242,648	236,552	236,000	552	0
69	243,664	235,536	235,000	536	0
70	244,680	234,520	234,000	520	0
71	245,696	233,504	233,000	504	0
72	246,712	232,488	232,000	488	0
73	247,728	231,472	231,000	472	0
74	248,744	230,456	230,000	456	0
75	249,760	229,440	229,000	440	0

快照區間	說明
n=50~59	顯示初期記憶體成長
n=60~65	到達記憶體高峰
n=66~75	顯示記憶體開始釋放

欄位名稱	說明
n	快照編號 snapshot 編號
time(B)	程式執行時累積的位元組（以 Byte 為單位）
total(B)	總堆積使用量（包含額外分配）
useful-heap(B)	真正由程式配置的記憶體量
extra-heap(B)	系統用來對齊、記錄等用途的額外記憶體
stacks(B)	call stack 區塊（這裡固定為 0）

觀察：

在本次實驗中，我們使用 Valgrind 的 Massif 工具來觀察程式執行過程中記憶體堆積區（heap）的使用變化。Massif 會根據記憶體變化量自動記錄「快照（snapshot）」，以時間為單位追蹤記憶體使用。根據 `ms_print massif.out.<pid>` 的結果，我們觀察到以下現象：

[1] 記憶體使用量呈階梯狀上升趨勢：

- 自 snapshot 編號 n=50 開始，記憶體使用量穩定成長。
- 每個快照之間的時間差約為 8,000 Bytes（從 time(B) 欄推算），代表系統每分配約 8KB 就會記錄一次快照。

[2] 總快照數至少為 76 個：

- 圖表中顯示至 n=75，代表此執行共產生了 76 個快照。
- 這表示 Massif 已完整記錄程式從啟動到記憶體釋放的整個階段。

[3] 總體記憶體使用量（total heap）趨勢清楚：

- 使用量從 183,000 Bytes 緩步上升，最終達到 239,000 Bytes。
- 達到高峰後記憶體釋放，因此出現總量下降的現象，代表程式有正常釋放記憶體。

2.[5%] Then, point out how many bytes are allocated and used at peak respectively.

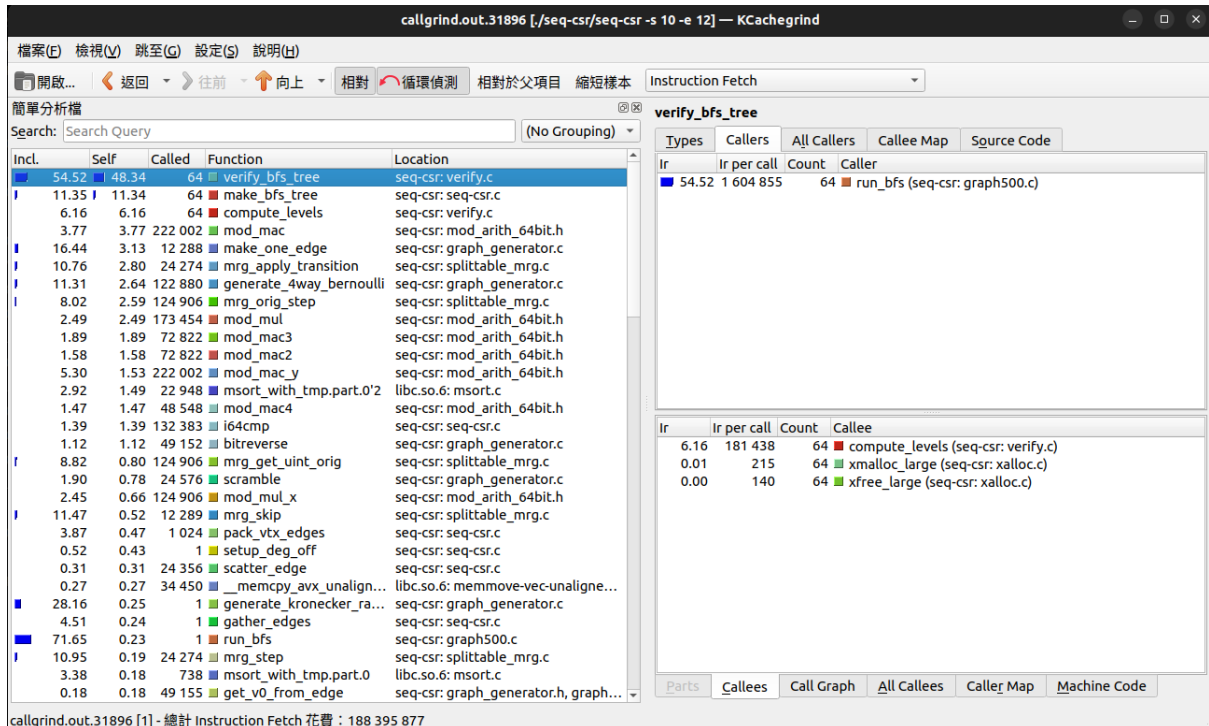
n	time(B)	total(B)	useful-heap(B)	extra-heap(B)
65	239,600	239,600	239,000	600

峰值時的記憶體使用量分析：

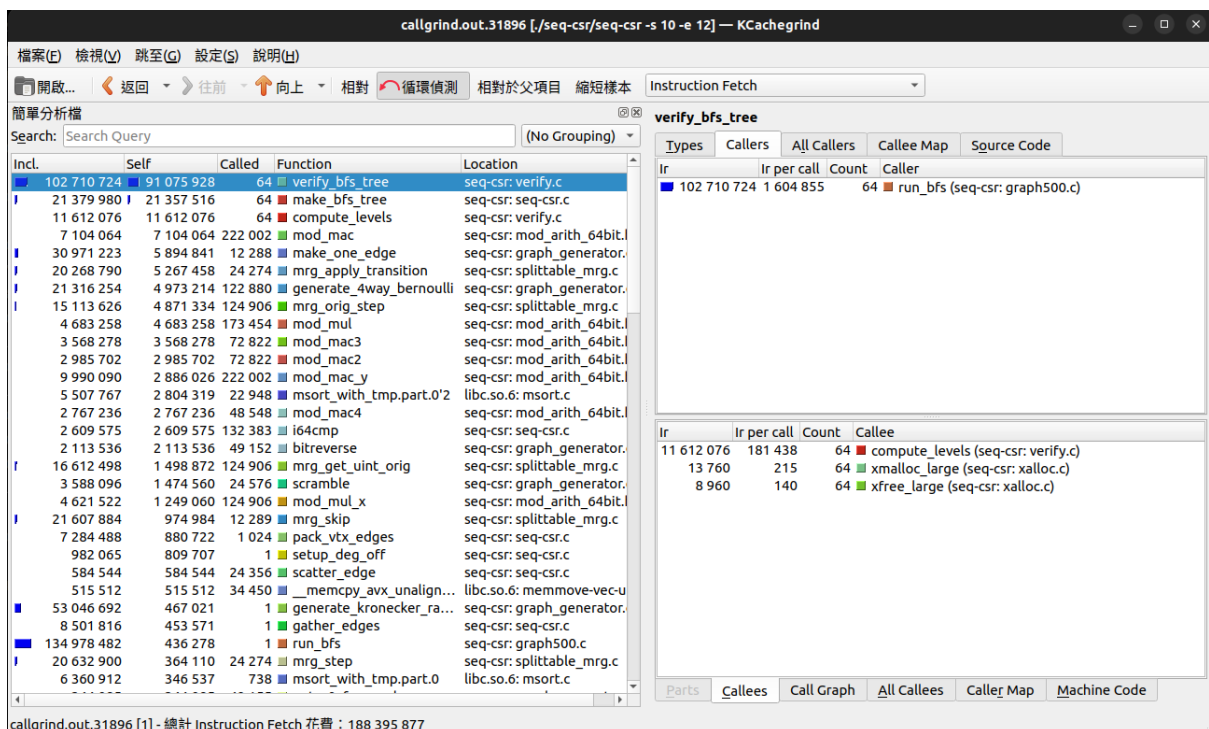
- 峰值時間點：snapshot n=65
- 總配置記憶體（total heap）：239,600 Bytes
- 實際有效使用（useful heap）：239,000 Bytes
- 系統額外分配（extra heap）：600 Bytes（管理結構、對齊等用途）

4. Callgrind - [20%]

1. [10%] Please use kcachegrind GUI to indicate which function is most expensive in terms of time (excluding the time of their callee functions). Please include a screenshot of the call graph.



相對指令數(%)



絕對指令數(條)

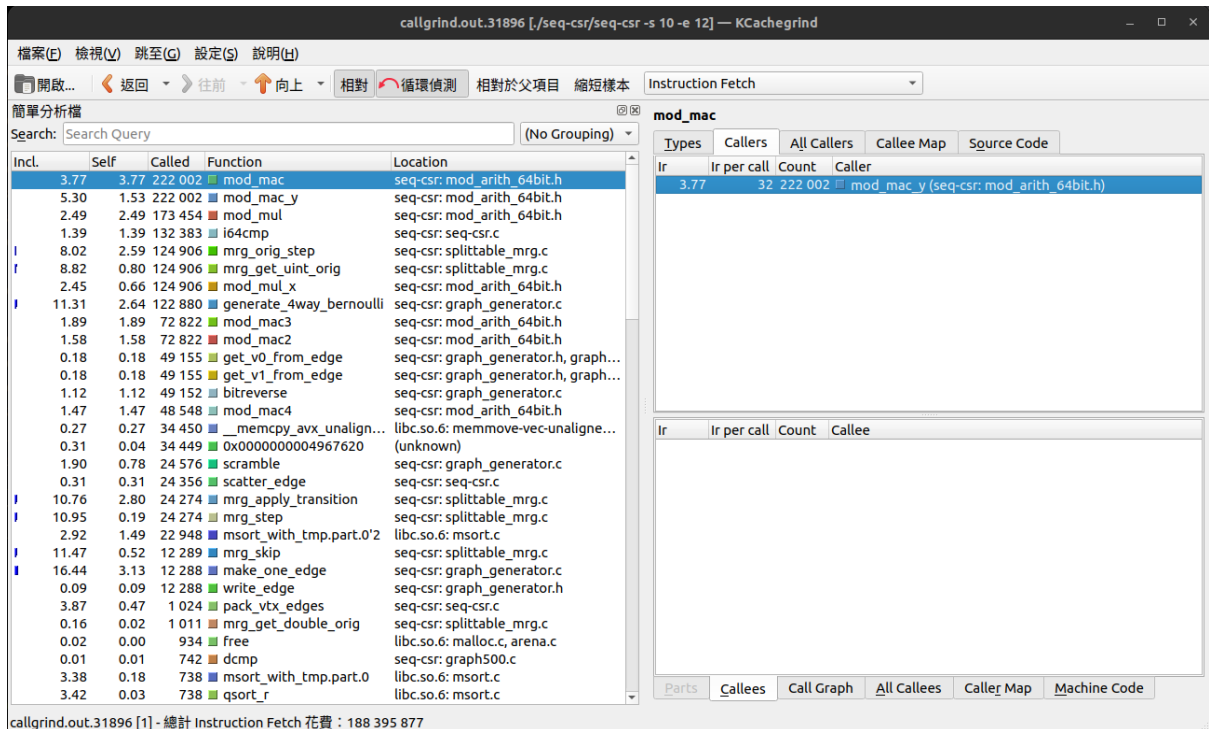
分析：

從左側函數列表可知，verify_bfs_tree() 函數在執行中「自身耗費時間（Self cost）」最多：

- 函數名稱：verify_bfs_tree
- 來源檔案：seq-csr/verify.c
- Self 指令數（Self Ir）：48.34%，約 91,075,928 條指令
- 總指令數（Inclusive Ir）：54.52%，含其呼叫的其他函數

這表示該函數本身的運算負擔最高，是效能最主要的瓶頸之一。

2. [10%] Point out which function is called most frequently, and identify its caller as well. Please include a screenshot of the call graph.



最常被呼叫函數

最常被呼叫的函數（依呼叫次數排序），從截圖可以看到：

- 函數名稱：mod_mac
- Self Time（自身耗時）：3.77%
- Call Count（被呼叫次數）：222,002 次
- 呼叫者函數（Caller）：mod_mac_y（來自 mod_arith_64bit.h）

這代表：

- mod_mac 是整體被呼叫次數最高的函數之一。
- 它由 mod_mac_y 呼叫，而且次數非常密集，表示在模數乘加操作中是重要的運算核心。

5. Pytorch profiler - [30%]

1. [15%] Please follow the tutorial to get the columns shown in the picture below.

```
ynd@valgrindenv:~/Desktop/Pytorch$ python3 test2.py
CPU
STAGE:2023-04-25 14:30:13 4634:4634 ActivityProfilerController.cpp:311] Completed Stage: Warm Up
STAGE:2023-04-25 14:30:13 4634:4634 ActivityProfilerController.cpp:317] Completed Stage: Collection
STAGE:2023-04-25 14:30:13 4634:4634 ActivityProfilerController.cpp:321] Completed Stage: Post Processing
```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	CPU Mem	Self CPU Mem	# of Calls
------	------------	----------	-------------	-----------	--------------	---------	--------------	------------

Please provide a screenshot of the analysis result, ensuring that the username and machinename are visible in the first line. Then, identify the top three functions in terms of CPU time excluding the time of their callee functions, except for the model label (e.g. `model_inference` in tutorials).

```
shihcw@shihcw-GF65-Thin-9SD:~/work_space/MSS/HW2$ python3 transformer_from_scratch.py
使用裝置：cpu
輸出 shape：torch.Size([2, 7, 10])
```

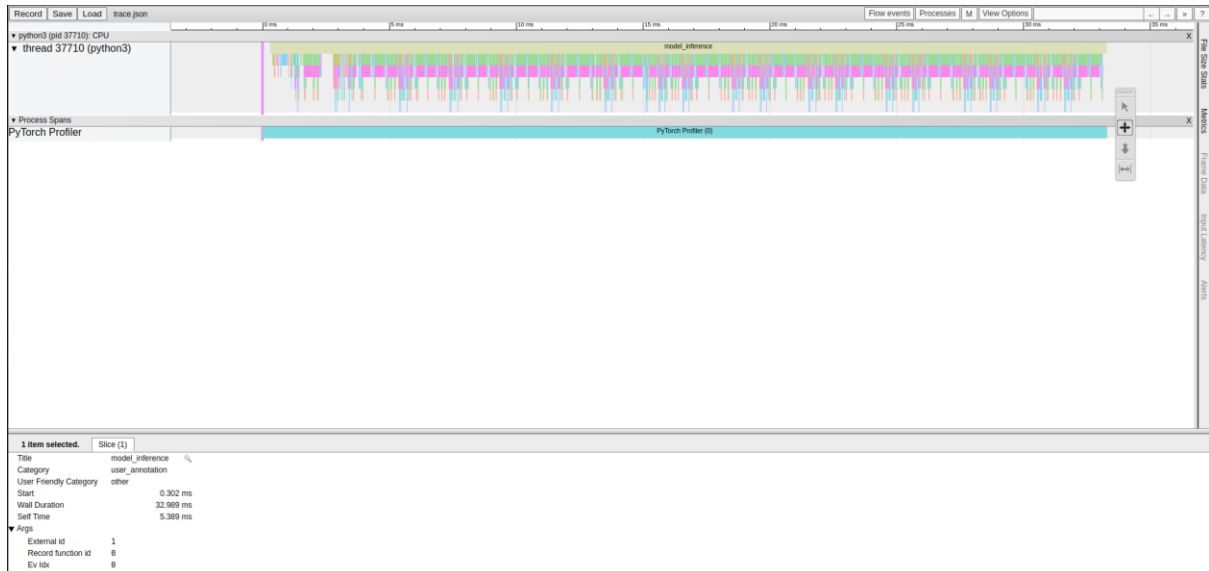
Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	# of Calls
aten::addmm	43.33%	14.294ms	49.23%	16.241ms	167.431us	97
model_inference	16.34%	5.389ms	100.00%	32.989ms	32.989ms	1
aten::copy_	9.68%	3.192ms	9.68%	3.192ms	14.124us	226
aten::einsum	2.66%	878.348us	14.42%	4.756ms	132.114us	36
aten::view	2.64%	871.273us	2.64%	871.273us	2.162us	403
aten::clamp_min	2.06%	680.332us	2.06%	680.332us	56.694us	12
aten::bmm	2.01%	663.003us	2.04%	671.955us	18.665us	36
aten::linear	1.67%	551.382us	54.35%	17.931ms	184.854us	97
aten::empty	1.65%	545.082us	1.65%	545.082us	2.869us	190
aten::native_layer_norm	1.65%	543.200us	2.13%	703.080us	23.436us	30

Self CPU time total: 32.989ms

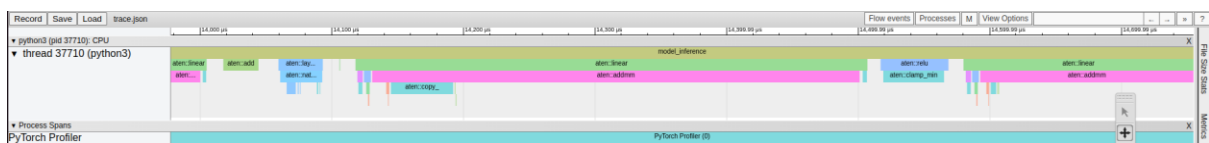
CPU 時間排名前三的函數（包含 `model_inference`，依照 Self CPU Time）：

排名	函數名稱	Self CPU 時間	Self CPU %	說明
1	<code>aten::addmm</code>	14.294 ms	43.33%	矩陣乘法並加總，常見於線性層 (Linear Layer)
2	<code>model_inference</code>	5.389 ms	16.34%	包含模型前向傳遞邏輯
3	<code>aten::copy_</code>	1.929 ms	5.86%	張量間複製操作，常見於資料傳輸

2. [15%] Output the profiling results to <output>.json and analyze in Chrome trace viewer. Take a screenshot of the visualization and point out which two functions (colors) appear the most (in terms of time), except for the model label (e.g. model_inference in tutorials).



放大後如下：



方法：利用 `chrome://tracing/` 進行分析
出現最多函數: `aten::addmm`(粉色), `aten::linear`(綠色)

結果來源	衡量指標	出現時間長的函數	原因分析
CLI 表格 (文字報告)	Self CPU only	<code>aten::addmm</code> , <code>aten::copy_</code>	它們自己計算邏輯密集，不依賴子函數
Chrome Trace 圖像	包含子函數的耗時	<code>aten::addmm</code> , <code>aten::linear</code>	<code>aten::linear</code> 包含 <code>addmm</code> 的整體耗時

問題背景：

- 根據命令列的 Profiler 表格結果，`aten::addmm` 和 `aten::copy_` 是除了 `model_inference` 外最佔用 Self CPU 的函數。
- 但在 Chrome Trace (<chrome://tracing>) 的視覺化時間軸上，顯得是 `aten::addmm` 和 `aten::linear` 最佔時間。

解釋與推論

1. `aten::linear` $\neq$ 單一 PyTorch op

- `aten::linear` 本身是一個封裝操作，底層通常會呼叫：
- `aten::addmm`（矩陣乘法與加總）
- 可能還有 `aten::bias_add`（或其他計算）

所以在 Chrome Trace 視覺化中看到 `aten::linear` 時，它實際代表的是：多個底層子運算（如 `addmm`）包裝而成的高層操作。

2. Profiler 表格中的 Self CPU 是純自己

- `aten::linear` 的 Self CPU 時間 幾乎為零或很小，因為它大部分時間花在呼叫 `aten::addmm`。
- `aten::copy_` 則是單一底層操作，自身就做事，因此它在表格中會顯得 Self CPU 高。

3. 視覺化 UI 的呈現偏向總時間

- Chrome Trace 的彩色區塊長短代表「整體耗時」，包含呼叫其他函數的時間（即 Total CPU）。
- 所以你看到的 `aten::linear` 時間條很長，並不是因為它自己耗很多 CPU，而是它呼叫了耗時的 `aten::addmm`。